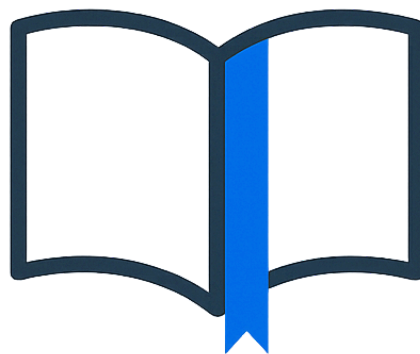


Università degli Studi di Salerno
Corso di Ingegneria del Software

BookMarker
Object Design Document
Versione 1.1



BookMarker

Data: 28/01/2026

Progetto: BookMarker	Versione: 1.1
Documento: Object Design Document	Data: 28/01/2026

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
Francesco Liguori	0512109651
Antonio Prisco	0512123502
Nicholas Franco Grimaldi	0512109702

Scritto da:	Tutti i partecipanti
--------------------	----------------------

Revision History

Data	Versione	Descrizione	Autore
19/12/2025	0.5	Prima stesura	Tutti i partecipanti
27/01/2025	1.0	Correzione Interfacce di classe e pacchetti	Nicholas Franco Grimaldi, Antonio Prisco
28/01/2026	1.1	Piccoli fix	Francesco Liguori

Indice

1. Introduzione	4
1.1 Riferimenti	4
1.2 Trade-Offs	4
1.2.1 Rapid Development vs Functionality	4
1.2.2 Robustness vs Complexity	4
1.2.3 Facilità d'uso vs Free-roaming	4
1.2.4 Funzionalità vs. Usabilità	4
1.2.5 Portabilità vs Efficienza	4
2. Pacchetti	5
3. Interfacce di Classe	6
3.1 Gestione Libri	6
3.2 Gestione Utenti	9
3.3 Gestione Prestiti	13
3.4 Gestione Recensioni	17
3.5 Gestione Segnalazioni	22

1. Introduzione

1.1 Riferimenti

Il seguente documento fa riferimento ai precedenti RAD e SDD.

1.2 Trade-Offs

1.2.1 Rapid Development vs Functionality

Per garantire il rilascio di una versione stabile del sistema entro le scadenze, sono state prioritizzate le funzionalità “Core” (Gestione Prestiti, Libri e Utenti in primo piano, Recensioni, Segnalazioni in secondo piano).

Abbiamo sacrificato funzionalità accessorie come ad esempio l’assegnazione di prestiti senza prenotazione (chi si presenta fisicamente in biblioteca) per concentrarci sulla solidità del ciclo di vita del prestito e della moderazione.

1.2.2 Robustness vs Complexity

Nonostante l'architettura sia mantenuta volutamente semplice, si è puntato sulla robustezza tramite un controllo rigoroso degli input.

Ogni metodo include blocchi di validazione manuale (es. controlli null, lunghezza stringhe) e gestione eccezioni try-catch. Questo aumenta la verbosità del codice ma riduce gli errori imprevisti.

1.2.3 Facilità d’uso vs Free-roaming

Il design è stato impostato per guidare l'utente passo dopo passo, limitando la sua libertà di azione per prevenire errori logici.

Invece di permettere una navigazione libera complessa, i flussi critici (come la prenotazione di un libro o l'inserimento di una recensione) sono lineari. Ad esempio, non è possibile recensire un libro se non è stato prima preso in prestito e letto (controllo nello storico prestiti).

1.2.4 Funzionalità vs. Usabilità

BookMarker non offre un numero elevato di funzionalità, in cambio di semplificare l'accesso alle informazioni e facilitare la consultazione dei libri e dei prestiti.

1.2.5 Portabilità vs Efficienza

Il sistema prevede portabilità a livello di sistema operativo (il codice è scritto in java) e a livello di interfaccia utente web dato che funziona su più browser e le interfacce sono Responsive. Un sistema progettato per essere altamente portabile, come un'applicazione Java, in determinate circostanze, può renderlo meno efficiente in

termini di velocità pura rispetto a un programma ottimizzato e compilato per un singolo ambiente specifico.

2. Pacchetti

L'implementazione del back-end di BookMarker adotta un'architettura a livelli basata sul pattern MVC (Model-View-Controller).

La struttura del codice sorgente (src/main/java/) è organizzata come segue:

In (it/bookmarker/):

- model: Contiene i JavaBeans (Entità) che mappano le tabelle del database (Utente, Libro, Prestito, Recensione, Segnalazione).
- dao: Contiene le interfacce e le classi che gestiscono la comunicazione diretta con il database tramite JDBC (Pattern DAO). In dao, /util: Contiene classi di utilità, come la gestione della connessione al Database (DBConnection).
- service: Contiene la logica di business e funge da intermediario tra i Controller e i DAO. Qui vengono applicate le regole di validazione e le trasformazioni dei dati.
- controller: Contiene le Servlet che gestiscono le richieste HTTP e indirizzano le risposte verso le viste (JSP).

In (/webapp):

- css: contiene i fogli di stile a cascata che definiscono il layout grafico e la presentazione dell'interfaccia utente.
- file.jsp: rappresentano le Viste (View) del sistema: si occupano di generare dinamicamente l'HTML da inviare al client, visualizzando i dati (attributi) passati dai Controller.

Di seguito vengono specificati i contratti dei principali metodi delle classi Service, che rappresentano il cuore logico dell'applicazione.

3. Interfacce di Classe

3.1 Gestione Libri

Non sono presenti invarianti.

Nome metodo	+aggiungiLibro(String titolo, String autore, String genere, String copieStr, String dataPubStr, String copertina, String descrizione)
Descrizione	Crea una nuova istanza di Libro, valida i formati e la inserisce nel catalogo.
Pre-condizioni	context LibroService::aggiungiLibro(titolo, autore, genere, copieStr, dataPubStr, copertina, descrizione) pre: titolo <> null and autore <> null and genere <> null and copertina <> null and descrizione <> null and copieStr.toInteger() >= 0 and dataPubStr.toDate() <= currentTime()
Post-condizioni	context LibroService::aggiungiLibro(titolo, autore, genere, copieStr, dataPubStr, copertina, descrizione) post: self.libri->exists(l l.titolo = titolo and l.autore = autore and l.disponibilita = copieStr.toInteger() and l.dataPubblicazione = dataPubStr.toDate()))

Nome metodo	+aggiornaDisponibilita(String idStr, String quantitaStr)
Descrizione	Aggiorna il numero di copie disponibili per un determinato libro.

Pre-condizioni	context LibroService::aggiornaDisponibilita(idStr, quantitaStr) pre: self.libri->exists(l l.id = idStr.toInteger()) and idStr.toInteger() <> null and quantitaStr.toInteger() >= 0
Post-condizioni	context LibroService::aggiornaDisponibilita(idStr, quantitaStr) post: self.libri->exists(l l.id = idStr.toInteger() and l.disponibilita = quantitaStr.toInteger())

Nome metodo	+rimuoviLibro(String idStr) : String
Descrizione	Rimuove un libro dal catalogo.
Pre-condizioni	context LibroService::rimuoviLibro(idStr) : String pre: self.libri->exists(l l.id = idStr.toInteger()) and !self.prestiti->exists(p p.libro.id = idStr.toInteger() and p.stato <> StatoPrestito::RESTITUITO)
Post-condizioni	context LibroService::rimuoviLibro(idStr) : String post: !self.libri->exists(l l.id = idStr.toInteger()) and result = null

Nome metodo	+getCatalogoCompleto() : List<Libro>
-------------	--------------------------------------

Descrizione	Restituisce la lista completa dei libri presenti nel catalogo.
Pre-condizioni	context LibroService::getCatalogoCompleto() : List<Libro> pre: true
Post-condizioni	context LibroService::getCatalogoCompleto() : List<Libro> post: result = self.libri

Nome metodo	+getDettaglioLibro(id: Integer) : Libro
Descrizione	Recupera i dettagli di un singolo libro tramite il suo identificativo.
Pre-condizioni	context LibroService::getDettaglioLibro(id: Integer) : Libro pre: self.libri->exists(l l.id = id)
Post-condizioni	context LibroService::getDettaglioLibro(id: Integer) : Libro post: self.libri->exists(l l.id = id and result = l)

3.2 Gestione Utenti

Nome metodo	+registraUtente(String nome, String cognome, String cf, String email, String password, String confirmPassword, String domanda, String risposta)
Descrizione	Valida i dati anagrafici e di sicurezza, verifica l'unicità di email e codice fiscale e registra un nuovo utente nel sistema.
Pre-condizioni	context UtenteService::registraUtente(nome, cognome, cf, email, password, confirmPassword, domanda, risposta) pre: nome.matches(NOME_PATTERN) and cognome.matches(NOME_PATTERN) and cf.matches(CF_PATTERN) and email.contains("@") and email.contains(".") and password.matches(PASS_PATTERN) and password = confirmPassword and domanda <> null and risposta <> null and !self.utenti->exists(u u.email = email or u.cf = cf)
Post-condizioni	context UtenteService::registraUtente(nome, cognome, cf, email, password, confirmPassword, domanda, risposta) post: self.utenti->exists(u u.email = email and u.cf = cf and u.domandaSicurezza = domanda and u.rispostaSicurezza = risposta and u.password <> null and u.stato = StatoUtente::IN_ATTESA)

Nome metodo	+login(String email, String password) : Utente
Descrizione	Verifica le credenziali e lo stato dell'account. Permette l'accesso solo se la password corrisponde e l'utente non è in attesa di approvazione.

Pre-condizioni	context UtenteService::login(email, password) : Utente pre: email <> null and password <> null
Post-condizioni	context UtenteService::login(email, password) : Utente post: (self.utenti->exists(u u.email = email and u.password.matches(password) and u.stato <> StatoUtente::IN_ATTESA and result = u)) or result = null

Nome metodo	+accettaUtente(String email)
Descrizione	Il bibliotecario approva la registrazione di un utente, portando il suo stato da "in_attesa" ad "attivo".
Pre-condizioni	context UtenteService::accettaUtente(email) pre: self.utenti->exists(u u.email = email and u.stato = StatoUtente::IN_ATTESA)
Post-condizioni	context UtenteService::accettaUtente(email) post: self.utenti->exists(u u.email = email and u.stato = StatoUtente::ATTIVO)

Nome metodo	+rifiutaUtente(String email)
Descrizione	Il bibliotecario rifiuta la richiesta di registrazione di un utente, rimuovendolo dal database.

Pre-condizioni	context UtenteService::rifiutaUtente(email) pre: self.utenti->exists(u u.email = email and u.stato = StatoUtente::IN_ATTESA)
Post-condizioni	context UtenteService::rifiutaUtente(email) post: !self.utenti->exists(u u.email = email)

Nome metodo	+resetPassword(String email, String rispostaSicurezza, String nuovaPassword, String confermaPassword)
Descrizione	Permette il ripristino della password verificando la risposta alla domanda di sicurezza preimpostata.
Pre-condizioni	context UtenteService::resetPassword(email, rispostaSicurezza, nuovaPassword, confermaPassword) pre: self.utenti->exists(u u.email = email and u.rispostaSicurezza.toUpperCase() = rispostaSicurezza.toUpperCase()) and nuovaPassword.matches(PASS_PATTERN) and nuovaPassword = confermaPassword
Post-condizioni	context UtenteService::resetPassword(email, rispostaSicurezza, nuovaPassword, confermaPassword) post: self.utenti->exists(u u.email = email and u.password.matches(nuovaPassword))

Nome metodo	+getUtentiDaApprovare() : List<Utente>
-------------	--

Descrizione	Restituisce la lista di tutti gli utenti che hanno completato la registrazione ma sono ancora in attesa di approvazione.
Pre-condizioni	context UtenteService::getUtentiDaApprovare() : List<Utente> pre: true
Post-condizioni	context UtenteService::getUtentiDaApprovare() : List<Utente> post: result = self.utenti->select(u u.stato = StatoUtente::IN_ATTESA)

Nome metodo	+recuperaDomanda(String email) : String
Descrizione	Recupera il testo della domanda di sicurezza associata all'email fornita per avviare la procedura di recupero password.
Pre-condizioni	context UtenteService::recuperaDomanda(email) : String pre: self.utenti->exists(u u.email = email)
Post-condizioni	context UtenteService::recuperaDomanda(email) : String post: self.utenti->exists(u u.email = email and result = u.domandaSicurezza)

3.3 Gestione Prestiti

Nome metodo	+prenotaLibro(emailUtente: String, idLibroStr: String, dataRitiroStr: String)
Descrizione	Crea una nuova richiesta di prestito dopo aver validato formato dati, disponibilità date, limite prestiti attivi dell'utente e duplicati.
Pre-condizioni	<pre>context PrestitoService::prenotaLibro(emailUtente, idLibroStr, dataRitiroStr) pre: emailUtente <> null and idLibroStr.toInteger() > 0 and dataRitiroStr.toDate() >= currentTime() and dataRitiroStr.toDate() <= currentTime() + 2 and self.prestiti->select(p p.utente = emailUtente and p.stato = StatoPrestito::RITIRATO).size() < 3 and !self.prestiti->exists(p p.utente = emailUtente and p.libro.id = idLibroStr.toInteger() and (p.stato = StatoPrestito::RICHIESTO or p.stato = StatoPrestito::RITIRATO))</pre>
Post-condizioni	<pre>context PrestitoService::prenotaLibro(emailUtente, idLibroStr, dataRitiroStr) post: self.prestiti->exists(p p.utente = emailUtente and p.libro.id = idLibroStr.toInteger() and p.dataRitiro = dataRitiroStr.toDate() and p.stato = StatoPrestito::RICHIESTO)</pre>

Nome metodo	+approvaRichiestaPrestito(idStr: String)
Descrizione	Il gestore approva una richiesta di prestito, passando lo stato da "Richiesto" a "prenotato", verificando la disponibilità di copie.

Pre-condizioni	<pre>context PrestitoService::approvaRichiestaPrestito(idStr) pre: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::RICHIESTO and self.libri->exists(l l.id = p.libroId and l.disponibilita > 0))</pre>
Post-condizioni	<pre>context PrestitoService::approvaRichiestaPrestito(idStr) post: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::PRENOTATO)</pre>

Nome metodo	<code>+rifiutaRichiestaPrestito(idStr: String, motivazione: String) : Boolean</code>
Descrizione	Il gestore rifiuta una richiesta di prestito inserendo una motivazione.
Pre-condizioni	<pre>context PrestitoService::rifiutaRichiestaPrestito(idStr, motivazione) : Boolean pre: self.prestiti->exists(p p.id = idStr.toInteger())</pre>
Post-condizioni	<pre>context PrestitoService::rifiutaRichiestaPrestito(idStr, motivazione) : Boolean post: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::ANNULLATO and p.motivazione = motivazione) and result = true</pre>

Nome metodo	+confermaRitiro(String idStr)
Descrizione	Conferma l'avvenuto ritiro del libro da parte dell'utente, attivando il prestito (cambio stato da "prenotato" a "In Corso").
Pre-condizioni	context PrestitoService::confermaRitiro(idStr) pre: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::PRENOTATO)
Post-condizioni	context PrestitoService::confermaRitiro(idStr) post: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::IN_CORSO)

Nome metodo	+annullaPrestito(idStr: String, motivazione: String)
Descrizione	Annulla un prestito (richiesto o prenotato). Se il libro era già stato prenotato, ripristina la disponibilità nel catalogo.
Pre-condizioni	context PrestitoService::annullaPrestito(idStr, motivazione) pre: motivazione.size() >= 10 and self.prestiti->exists(p p.id = idStr.toInteger() and (p.stato = StatoPrestito::RICHIESTO or p.stato = StatoPrestito::PRENOTATO))

Post-condizioni	<pre>context PrestitoService::annullaPrestito(idStr, motivazione) post: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::ANNULLATO and (p.stato@pre = StatoPrestito::PRENOTATO implies p.libro.disponibilita = p.libro.disponibilita@pre + 1))</pre>
-----------------	---

Nome metodo	+registraRestituzione(idStr: String)
Descrizione	Registra la restituzione di un libro, terminando il prestito e incrementando la disponibilit� del libro nel catalogo.
Pre-condizioni	<pre>context PrestitoService::registraRestituzione(idStr) pre: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::IN_CORSO)</pre>
Post-condizioni	<pre>context PrestitoService::registraRestituzione(idStr) post: self.prestiti->exists(p p.id = idStr.toInteger() and p.stato = StatoPrestito::RESTITUITO and p.libro.disponibilita = p.libro.disponibilita@pre + 1)</pre>

Nome metodo	+getRichiesteInAttesa() : List<Prestito>
Descrizione	Recupera la lista di tutti i prestiti che sono nello stato "Richiesto".

Pre-condizioni	<pre>context PrestitoService::getRichiesteInAttesa() : List<Prestito></pre> <pre>pre: true</pre>
Post-condizioni	<pre>context PrestitoService::getRichiesteInAttesa() : List<Prestito></pre> <pre>post: result = self.prestiti->select(p p.stato = StatoPrestito::RICHIESTO)</pre>

Nome metodo	<code>+getStoricoUtente(email: String) : List<Prestito></code>
Descrizione	Recupera l'intero storico dei prestiti associati a uno specifico utente.
Pre-condizioni	<pre>context PrestitoService::getStoricoUtente(email) : List<Prestito></pre> <pre>pre: email <> null</pre>
Post-condizioni	<pre>context PrestitoService::getStoricoUtente(email) : List<Prestito></pre> <pre>post: result = self.prestiti->select(p p.utente = email)</pre>

3.4 Gestione Recensioni

Nome metodo	<code>+aggiungiRecensione(emailUtente: String, idLibroStr: String, testo: String, votoStr: String) : Boolean</code>
-------------	---

Descrizione	Valida i dati in input (non nulli e formato corretto) e salva una nuova recensione associata all'utente e al libro specificato.
Pre-condizioni	context RecensioneService::aggiungiRecensione(emailUtente, idLibroStr, testo, votoStr) : Boolean pre: emailUtente <> null and idLibroStr <> null and votoStr <> null and votoStr.toInteger() >= 1 and votoStr.toInteger() <= 5 and !self.recensioni->exists(r r.utente = emailUtente and r.libroId = idLibroStr.toInteger())
Post-condizioni	context RecensioneService::aggiungiRecensione(emailUtente, idLibroStr, testo, votoStr) : Boolean post: self.recensioni->exists(r r.utente = emailUtente and r.libroId = idLibroStr.toInteger() and r.testo = testo and r.voto = votoStr.toInteger()) and result = true

Nome metodo	+getRecensioniPubbliche(idLibro: Integer) : List<Recensione>
Descrizione	Restituisce la lista delle sole recensioni attive/visibili per un determinato libro (vista utente).
Pre-condizioni	context RecensioneService::getRecensioniPubbliche(idLibro) : List<Recensione> pre: true
Post-condizioni	context RecensioneService::getRecensioniPubbliche(idLibro) : List<Recensione> post: result = self.recensioni->select(r r.libroId = idLibro and r.visibile = true)

Nome metodo	+getRecensioniPerModeratore(idLibro: Integer) : List<Recensione>
Descrizione	Restituisce tutte le recensioni associate a un libro, incluse quelle nascoste o segnalate (vista moderatore).
Pre-condizioni	context RecensioneService::getRecensioniPerModeratore(idLibro) : List<Recensione> pre: true
Post-condizioni	context RecensioneService::getRecensioniPerModeratore(idLibro) : List<Recensione> post: result = self.recensioni->select(r r.libroId = idLibro)

Nome metodo	+getMappaRecensioniPerStorico(email: String, storico: List<Prestito>) : Map<Integer, Recensione>
Descrizione	Costruisce una mappa che associa l'ID del libro alla recensione dell'utente, basandosi sullo storico dei prestiti.
Pre-condizioni	context RecensioneService::getMappaRecensioniPerStorico(email: String, storico: List<Prestito>) : Map pre: email <> null and storico <> null

Post-condizioni	<pre>context RecensioneService::getMappaRecensioniPerStorico(email: String, storico: List<Prestito>) : Map post: storico->select(p p.isRecensito = true)->forall(p result.get(p.libroId) = self.recensioni->select(r r.utente = email and r.libroId = p.libroId))</pre>
-----------------	---

Nome metodo	<code>+deleteRecensioneUtente(emailUtente: String, idLibroStr: String) : boolean</code>
Descrizione	Permette a un utente di eliminare la propria recensione per un libro specifico.
Pre-condizioni	<pre>context RecensioneService::deleteRecensioneUtente(emailUtente: String, idLibroStr: String) : boolean pre: emailUtente <> null and idLibroStr <> null and self.recensioni->exists(r r.utente = emailUtente and r.libroId = idLibroStr.toInteger())</pre>
Post-condizioni	<pre>context RecensioneService::deleteRecensioneUtente(emailUtente: String, idLibroStr: String) : boolean post: !self.recensioni->exists(r r.utente = emailUtente and r.libroId = idLibroStr.toInteger()) and result = true</pre>

Nome metodo	<code>+deleteRecensioneModeratore(idRecensioneStr: String) : boolean</code>
Descrizione	Permette a un moderatore di eliminare una qualsiasi recensione.

Pre-condizioni	context RecensioneService::deleteRecensioneModeratore(idRecensioneStr: String) : boolean pre: idRecensioneStr <> null and self.recensioni->exists(r r.id = idRecensioneStr.toInteger())
Post-condizioni	context RecensioneService::deleteRecensioneModeratore(idRecensioneStr: String) : boolean post: !self.recensioni->exists(r r.id = idRecensioneStr.toInteger()) and result = true

Nome metodo	+impostaVisibilita(idStr: String, visibile: Boolean) : Boolean
Descrizione	Modifica lo stato di visibilità di una recensione (nascondi/mostra).
Pre-condizioni	context RecensioneService::impostaVisibilita(idStr: String, visibile: Boolean) : Boolean pre: idStr <> null and self.recensioni->exists(r r.id = idStr.toInteger())
Post-condizioni	context RecensioneService::impostaVisibilita(idStr: String, visibile: Boolean) : Boolean post: self.recensioni->exists(r r.id = idStr.toInteger() and r.visibile = visibile) and result = true

3.5 Gestione Segnalazioni

Nome metodo	+segnalaRecensione(idRecensioneStr: String, emailUtente: String, motivo: String) : String
Descrizione	Crea una nuova segnalazione per una recensione, verificando che il motivo rispetti la lunghezza minima richiesta.
Pre-condizioni	context SegnalazioneService::segnalaRecensione(idRecensioneStr: String, emailUtente: String, motivo: String) : String pre: idRecensioneStr <> null and emailUtente <> null and motivo <> null and motivo.trim().size() >= 20
Post-condizioni	context SegnalazioneService::segnalaRecensione(idRecensioneStr: String, emailUtente: String, motivo: String) : String post: self.segnalazioni->exists(s s.recensioneId = idRecensioneStr.toInteger() and s.utente = emailUtente and s.motivo = motivo) and result = "successo"

Nome metodo	+getTutteLeSegnalazioni() : List<Segnalazione>
Descrizione	Restituisce la lista completa delle segnalazioni presenti nel sistema per la visualizzazione da parte del moderatore.
Pre-condizioni	context SegnalazioneService::getTutteLeSegnalazioni() : List<Segnalazione> pre: true

Post-condizioni	<pre>context SegnalazioneService::getTutteLeSegnalazioni() : List<Segnalazione></pre> <pre>post: result = self.segnalazioni</pre>
-----------------	---

Nome metodo	<code>+chiudiSegnalazione(idStr: String, note: String) : boolean</code>
Descrizione	Aggiorna lo stato di una segnalazione indicandola come chiusa e salvando le note del moderatore.
Pre-condizioni	<pre>context SegnalazioneService::chiudiSegnalazione(idStr: String, note: String) :</pre> <pre>boolean</pre> <pre>pre: idStr <> null and idStr.size() > 0 and self.segnalazioni->exists(s s.id =</pre> <pre>idStr.toInteger())</pre>
Post-condizioni	<pre>context SegnalazioneService::chiudiSegnalazione(idStr: String, note: String) :</pre> <pre>boolean</pre> <pre>post: self.segnalazioni->exists(s s.id = idStr.toInteger()) and s.stato =</pre> <pre>StatoSegnalazione::CHIUSA and ((note = null implies s.note = "") and (note <> null</pre> <pre>implies s.note = note))) and result = true</pre>